

Trabalho Prático 1: Commerce TechGear

Disciplina: Programação Orientada a Objetos

{*Entregar até o dia 16/04}

Prof. Dr. Lucas C. Ribas

Sumário

1	Commerce TechGear	1
2	Diagrama UML de Classes	1
3	Banco de Dados	4
4	Avaliação	5
5	Dúvidas	5

1 Commerce TechGear

Neste projeto, você desenvolverá um sistema de Commerce utilizando Java, que possibilitará a criação e gestão de diversas classes fundamentais para o funcionamento de uma loja. Entre estas, destacam-se `DisplayLoja`, `Loja`, `Categoria`, `ProdutoFisico` e `ProdutoVirtual`. O sistema será projetado para ser operado por funcionários do Commerce, otimizando o processo de venda dos produtos.

A arquitetura do sistema gira em torno da classe `Loja`, que agrega múltiplas instâncias de `Categoria`. Cada `Categoria` pode conter um ou mais produtos, que são representados por duas classes distintas: `ProdutoFisico` e `ProdutoVirtual`. Essas classes de produtos armazenam informações cruciais sobre os itens à venda, tais como preço, nome, descrição, identificação do produto, marca e outras características pertinentes. Para efetivar a implementação deste sistema, é necessário desenvolver as classes `DisplayLoja`, `Categoria`, `Loja`, `ProdutoFisico` e `ProdutoVirtual`, atendendo aos requisitos mínimos previstos (campos e métodos descritos para cada uma). É encorajado expandir a funcionalidade do sistema por meio da adição de métodos, campos e classes adicionais.

No início do programa, a função `main` deve definir três variáveis do tipo `String`, contendo os caminhos para os arquivos: `categorias.txt`, `produtoFisico.txt` e `produtoVirtual.txt`. Instruções detalhadas sobre o acesso e a utilização desses arquivos estão disponíveis na Seção 3 deste documento. Após a configuração inicial na função `main`, seu código deve criar uma instância de `DisplayLoja`, passando as três `Strings` com os caminhos dos arquivos de dados ao construtor. A classe `DisplayLoja` irá carregar os dados e disponibilizar a interface (linha de comando) para comunicação com o usuário, com busca de produtos, compra, inclusão e remoção, etc.

2 Diagrama UML de Classes

O diagrama UML apresentado na Figura 1 descreve as classes, atributos e métodos do sistema de Commerce.

IMPORTANTE:

- Embora as classes apresentadas sejam obrigatórias, outras classes podem ser criadas conforme necessário.
- Da mesma forma, os métodos e atributos listados são sugestões (mínimo que deve ter), vocês podem fazer modificações e incluir operações extras. Por exemplo, não foram especificados os métodos `set` e `get`. Também fiquem a vontade para ajustar e modificar os parâmetros e retorno dos métodos caso seja necessário.
- O usuário do sistema é o operador do Commerce.
- No lugar de lista (que é um conceito ainda não visto em aula), podem usar vetores e matrizes.

Seguem algumas informações das classes.

- **Produto (Classe Abstrata)**

- Armazena atributos comuns a todos os produtos, como ID, nome, preço, descrição e marca.
- Possui métodos para atualizar o estoque e o preço dos produtos.
- Não pode ser instanciada diretamente devido à sua natureza abstrata.

- **ProdutoFísico**

- Herda atributos da classe abstrata Produto.
- Adiciona atributos específicos de produtos físicos, como peso e dimensões.
- Inclui um método para calcular o frete baseado em suas características físicas.

- **ProdutoVirtual**

- Também herda atributos da classe abstrata Produto.
- Adiciona atributos específicos de produtos virtuais, como o tamanho do arquivo e o formato.
- Contém um método para realizar o download do produto virtual.

- **DisplayLoja**

- Atua como a interface com o usuário, permitindo interações com a loja.
- Possibilita a visualização de produtos, adição de produtos ao carrinho e realização de compras.
- Permite a gestão de categorias e produtos através de métodos específicos.

- **Loja**

- Contém informações da loja e mantém a lista de categorias e produtos.
- Permite adicionar e remover categorias e produtos, bem como buscar por eles.

- **Categoria**

- Representa uma categoria de produtos dentro da loja.
- Mantém uma lista de produtos e fornece métodos para gerenciar essa lista.

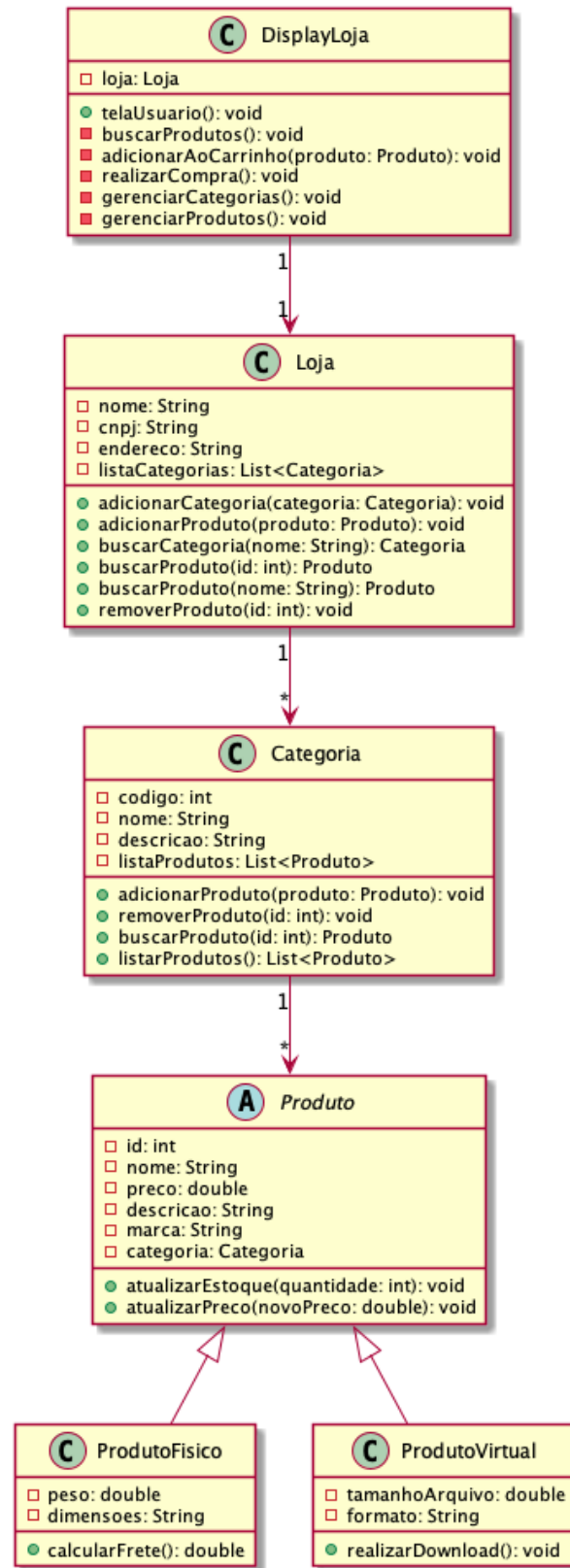


Figura 1: Diagrama UML do projeto.

3 Banco de Dados

Após implementar todas as classes e funcionalidades, você poderá testar o seu programa usando os dados fornecidos em 3 arquivos. A primeira linha nos arquivos contém qual o nome de cada campo.

- `categorias.txt`: esse arquivo possui linhas com os campos de cada categoria. Campos separados pelo caractere “#”
- `produtoVirtual.txt`: esse arquivo possui uma lista de produtos virtuais, onde cada linha representa um produto com seus campos separados pelo caractere “#”
- `produtoFisico.txt`: esse arquivo possui uma lista de produtos físicos, onde cada linha representa um produto com seus campos separados pelo caractere “#”.

Note que é possível ligar os produtos com as categorias por meio do número/código da categoria que está presente em ambos os arquivos. Por exemplo, olhando para os dados do produto abaixo, é possível ver que esse produto pertence à categoria de código 115 (Jogos e Consoles).

```
1 61535 # Jogo de Estrat gia Espacial # 293.94 # Comande uma civiliza o  
no espa o e conquiste a gal xia # Mann-Roberson # 115 # 4.36 GB # APP
```

Para ler os arquivos de dados, use o seguinte código abaixo. Note que, na linha 18 do código, a linha que foi lida do arquivo é quebrada em campos usando “#” como o separador. Portanto, o vetor de String `campos` terá todos os campos, um em cada posição. Outra observação é que os campos que são numéricos precisam ser convertidos de String para double (ou int e long). Para isso, é só usar os métodos `valueOf` das classes dos tipos primitivos, por exemplo: `int valorConvertido = Integer.valueOf(str);`

```
1 // Cria um objeto File com o caminho do arquivo  
2 File arquivo = new File("../contas.txt");  
3  
4 // Cria um objeto Scanner para ler o arquivo  
5 Scanner scanner = null;  
6 try {  
7     scanner = new Scanner(arquivo);  
8 } catch (FileNotFoundException ex) {  
9     //Disciplinapoo o nome da class onde est o c digo  
10    Logger.getLogger(Disciplinapoo.class.getName()).log(Level.SEVERE, null,  
ex);  
11 }  
12  
13 // L cada linha do arquivo  
14 while (scanner.hasNextLine()) {  
15     String linha = scanner.nextLine();  
16  
17     // Divide a linha em campos separados por #  
18     String[] campos = linha.split( "#");  
19  
20     // Imprime o conte do de cada campo (pode usar para testar se a  
leitura est correta, nesse caso ele imprime com | como separador)  
21     for (String campo : campos) {  
22         System.out.print(campo + " | ");  
23     }  
24     // Pula uma linha  
25     System.out.println();  
26 }
```

4 Avaliação

- Este trabalho é **individual**. Isso significa que qualquer tipo de plágio será penalizado com nota **ZERO**.
- Posteriormente à entrega do trabalho, poderei solicitar que alguns de vocês expliquem o código. Essa entrevista também poderá influenciar na nota final.
- Implementações extras e a utilização de boas práticas de programação, como documentação e modularização, serão levados em consideração na avaliação.
- A entrega deve ser feita via classroom. Você deverá anexar todo o projeto em formato `Nome_RA.zip`.
- **Deadline: 16/04/2024**. O sistema irá fechar às 23:59h, não recebendo mais submissões após o prazo. Não aceitarei trabalhos entregues via e-mail ou por outro meio.

5 Dúvidas

Meu e-mail é `lucas.ribas@unesp.br` e meu gabinete é o 12 no DCCE (envie um e-mail antes para checar a disponibilidade).